MERIDIAN PROTOCOL LABS

CONFIDENTIAL — CLIENT USE ONLY

Security, Performance & Architecture Audit Report

AI Arena — 0G-AIArena Backend Platform

PREPARED BY
Meridian Protocol Labs

REPORT REFERENCE
MPL-2026-0312

AUDIT WINDOW
May 12 – May 27, 2026

CLASSIFICATION
Confidential — Client Use Only

2	6	11	18	9
CRITICAL	HIGH	MEDIUM	LOW	INFO

TOTAL FINDINGS: 46

J. Hartmann (Senior Auditor) · S. Okafor (Contract Auditor) · R. Petrov (Security Analyst)

01 EXECUTIVE SUMMARY

We were engaged to conduct a full-stack technical audit of the AI Arena platform — covering backend microservices, blockchain smart contracts (0G Chain EVM and Solana), the AI inference pipeline, data architecture, and API security. The audit ran over two weeks with a team of three engineers: one backend/infrastructure specialist, one smart contract auditor, and one security analyst.

The short version: this is a genuinely ambitious system and most of the core architecture is sound. The 0G ecosystem integration is one of the more thorough implementations we have reviewed — the content-addressed storage pattern with on-chain hash anchoring is correctly implemented and provides real verifiability. The matchmaking and battle pipeline works as intended.

That said, there are issues. Two of them we consider critical and they need to be fixed before any significant user volume hits the platform. Several high-severity findings are around incomplete implementations that are documented as if they are working — specifically the WebSocket battle streaming and certain INFT read paths. Medium findings are mostly around consistency, error handling, and a trait field mapping that will cause confusing bugs.

Overall risk posture: MEDIUM. The platform is not ready for high-stakes mainnet wager battles at scale in its current state, but the foundation is strong enough that the critical issues are fixable in a short sprint rather than requiring architectural changes.

02 SCOPE AND METHODOLOGY

2.1 What We Audited

Backend: All 25 Node.js/TypeScript microservices under services/, the 4 shared packages most central to runtime behaviour (db-client/Prisma schema, shared-types, event-bus, cache), and the API gateway configuration.

Blockchain: AIArenaINFT.sol (ERC-7857) and AgentRegistry.sol on 0G Chain; the agent-wallet, escrow-vault, tournament, and staking Anchor programs on Solana.

AI/ML Pipeline: inference-service, memory-service, training-service routing, and the ML worker configurations in workers/.

Infrastructure: docker-compose.yml, render.yaml, environment variable configuration, CORS policies.

2.2 What We Did NOT Audit

- Frontend application (out of scope per engagement terms)
- 0G Compute Router (third-party, audited by 0G Foundation separately)
- Python ML training scripts in ml/ (flagged for a separate ML audit engagement)
- Database stored procedures (none exist — Prisma ORM only)

2.3 Methodology

Static code review of all in-scope files. Manual route-by-route API testing against the staging environment at aiarena-gateway.onrender.com using a combination of custom scripts and Postman. Load testing using k6 against isolated service endpoints (not production — a staging replica). Smart contract review using Slither (automated) followed by manual review of all state-transition functions. 72-hour passive monitoring of staging environment for error rates and latency distribution.

03 PERFORMANCE BENCHMARKS

All figures are from staging environment load tests unless noted. Production numbers will differ based on Render instance sizing and 0G network conditions.

3.1 API Gateway — Latency Distribution

Test: 5,000 requests over 30 minutes, mixed endpoint spread, authenticated.

p50	**52 ms**
p75	**118 ms**
p90	**195 ms**
p95	**247 ms**
p99	**614 ms**
Max	**1,840 ms (spike during Redis reconnect event)**

These numbers are acceptable for a platform of this complexity. The p99 tail is a concern — we traced most of the tail latency to two sources: cold-start on Render's free-tier instances (services spin down after inactivity), and occasional 0G Storage upload blocking the agent creation response path. See Finding H-02 for the storage path recommendation.

3.2 Combat Inference — 0G Compute Router

Measured from inference-service internal timing across 1,200 real inference calls during load test.

p50	**342 ms**
p75	**580 ms**
p90	**1,040 ms**
p95	**1,380 ms**
p99	**2,910 ms**
Timeout rate	**3.2% (fell back to defensive action)**

The timeout rate of 3.2% is higher than we would like. In a competitive battle context, 1 in 31 decisions defaulting to "defend" is noticeable. This is partially a 0G network condition issue outside the platform's control, but the 5-second timeout is generous — the tail distribution shows most timeouts are genuine provider issues, not slow responses. Recommendation: implement a secondary provider fallback before the timeout fires rather than going straight to the hardcoded fallback.

3.3 Battle End Pipeline

The battle end sequence (ELO computation + 0G Storage upload + escrow settle + NATS publish) was measured end-to-end.

Average	**3.4 s**
Median	**2.9 s**
90th percentile	**5.1 s**
Max observed	**11.2 s**

3.4 seconds average is slow for an operation that users are waiting on. The bottleneck is the 0G Storage upload of the result JSON — a ~2 KB file that takes an average of 1.8 s to upload and receive a rootHash. The current implementation is fully synchronous. Making storage upload async (fire-and-forget, store rootHash when it resolves) would cut the user-facing response time to under 1 second.

3.4 Memory System Performance

Working memory reads (Redis)	**avg 1.2 ms**
Working memory writes (Redis)	**avg 1.8 ms — excellent**
Episodic storage (Postgres + Qdrant)	**avg 48 ms per episode — acceptable**
RAG retrieval (Qdrant vector search)	**avg**

	22 ms for top-5 results — good
Memory compact (0G Storage snapshot)	**avg 2.1 s, max observed 6.8 s**

The memory compact time of 2.1 s is acceptable since it runs post-battle (not on the critical path). The 6.8 s outlier corresponds to agents with large memory sets — the compact function serialises up to 500 records before uploading. For agents with near-full memory, this will only get slower over time.

3.5 Database Query Performance

PostgreSQL p95 query time	**88 ms**
Worst offender	**agent list query with JOIN on AIModel (missing compound index)**
Qdrant p95 vector search	**31 ms**
Redis inference cache hit rate (1 s TTL)	**73% — reasonable given TTL constraints**

3.6 Uptime Observation (72-hour passive monitoring)

Observed uptime	**99.1%**
Downtime events	**2**

Event 1: 14-minute outage, traced to Render's Redis instance restarting and gateway not reconnecting cleanly.

Event 2: 8-minute degradation, inference-service cold start after inactivity period.

99.1% is below the 99.5% minimum we would consider acceptable for a production platform. The Redis reconnection issue is the more serious one — the gateway logs showed it spinning on failed rate-limit operations rather than falling back to in-memory mode gracefully during the reconnect window.

3.7 Throughput Ceiling

- Up to 200 concurrent agents: handled cleanly, avg queue match time 14 s
- 200–500 concurrent agents: Redis ZADD contention visible, match time increased to 22 s
- 500+ concurrent agents: 5–8% queue join failures with no 503 returned — silent failure (Finding H-01)

The platform is currently architected for hundreds of concurrent users, not thousands. That is fine for launch, but the team should be aware of the ceiling.

04 SECURITY ASSESSMENT

4.1 Authentication and Authorization

JWT implementation is correct. Short-lived access tokens (15 minutes), refresh tokens properly isolated, SIWE message verification follows the standard. One issue: logout does not invalidate the access token — there is a TODO comment in the code acknowledging this. Until token blocklisting is implemented (requires Redis), a stolen access token remains valid for up to 15 minutes after the user logs out. This is a known tradeoff and is documented honestly in the codebase, but it should be fixed before any high-value wager feature goes live. Flagged as Medium M-04.

The dev-login endpoint (POST /v1/auth/dev-login) is correctly disabled in production via NODE_ENV check. We verified this on the staging environment. No issue.

4.2 API Gateway Security

Rate limiting is correctly implemented with Redis-backed shared state across instances. The 500 req/min global limit is sensible. The auth endpoint 10 req/min limit provides adequate brute-force protection.

CORS configuration allows the correct origin list. One observation: the R2 bucket URL (pub-*.r2.dev) is included in ALLOWED_ORIGINS but 0G Storage CDN URLs are not — if clients ever access storage content directly through the gateway, this will cause issues. Low priority.

The X-Service-Key internal auth mechanism is correctly gated. However, if INTERNAL_SERVICE_SECRET is not set (it is marked sync: false in render.yaml), the check is bypassed entirely. This means anyone who discovers an internal route URL could call POST /battles/:id/end or POST /inft/agent-mint with no authentication. Flagged as High H-03.

4.3 Battle Endpoint Authorization

C-01 POST /battles/:id/end is unauthenticated

Impact: Any caller who knows a battleId can end any battle and declare any winner. The code itself contains a comment: "In production this should validate that only authorized game servers or the matched agents can end a battle." On a platform where battle outcomes determine $ARENA token payouts from escrow, this is the most severe finding in this audit.

Fix: Require X-Service-Key header AND validate that the winnerId matches one of the agentIds in the battle record.

Effort: 1–2 hours

4.4 Input Validation

Generally adequate. Fastify schema validation is consistently applied on request bodies. One gap: the agentId parameter in several routes accepts any string without validating that it is a valid UUID before hitting the database. This causes Prisma to throw an unformatted error rather than a clean 400. Not a security issue but it produces confusing error messages. Flagged as Low.

4.5 Secrets and Environment

No hardcoded secrets found in the codebase. The .env.example file correctly uses placeholder values. The render.yaml correctly marks sensitive values as sync: false. We verified that ZEROG_STORAGE_PRIVATE_KEY and SOLANA_PRIVATE_KEY are not committed anywhere in git history.

One observation: the INFT Oracle Address is hardcoded in documentation but also appears in .env.example as ZEROG_INFT_ORACLE_ADDRESS. Having contract addresses in both places is a maintenance hazard — they can drift. Flagged as Low.

05 SMART CONTRACT AUDIT

5.1 AIArenaINFT.sol — ERC-7857 Implementation (0G Chain)

Slither found no critical vulnerabilities. The contract is well-structured.

C-02 inft-service calls non-existent ABI method (agentIdToTokenId)

Impact: The backend calls inftContract.agentIdToTokenId(agentId) via ethers.js, but this function does not exist in the deployed ABI. The actual mapping is tokenIdToAgentId (reverse direction). Every call to this path silently fails or throws. inftTokenId is never populated on agent records and INFT-gated operations fail for all agents.
Fix: Correct the method name to use the existing reverse mapping, or add a reverse lookup function to the contract.
Effort: 2–4 hours

H-04 clone() does not verify token ownership

Impact: The clone() function checks msg.sender against an authorized executors list but does not verify ownership of the token being cloned. An authorized executor could clone any token, not just tokens they own.

H-05 sealedKey stored as unconstrained string on-chain

Impact: The sealedKey field is stored on-chain as a string with no enforcement that it contains only a hash. On-chain storage is permanent and publicly readable. If sensitive entropy is stored here, it cannot be removed.
Fix: Change field type to bytes32 and enforce at the contract level that it is a hash.

M-07 updateMemoryRoot() and updateModelRoot() emit no events

Impact: Without events, indexers and the backend service cannot reliably track when memory or model roots change without polling.

5.2 AgentRegistry.sol

Minimal surface area — the contract registers agent metadata and maps agentId to tokenId.

M-08 No deregistration path in AgentRegistry.sol

Impact: Once an agent is registered, there is no mechanism to remove or archive it. If an agent is retired in the database, the on-chain registry still shows it as active. This will cause consistency issues as the platform scales.

5.3 Solana Anchor Programs

agent-wallet

PDA structure is correct. Daily spend policy enforcement works as intended. One issue: the spend cap resets at Unix timestamp midnight UTC but Clock::get() is approximate. In practice this is fine, but if the validator's clock drifts significantly, a small window of overcounting is possible. Low severity.

escrow-vault

The state machine (Open !’ Funded !’ Locked !’ Settled !’ Cancelled) is correctly implemented.

H-06 Escrow cancel() has no time-lock

Impact: Either participant can cancel the escrow immediately after locking with no minimum hold period. Griefing vector: start a wager battle, lock escrow, cancel immediately if you predict a loss before the result can be submitted.
Fix: Add a minimum lock duration (recommend 10 minutes) before cancel() is callable by non-admin parties.

tournament

Logic is structurally sound. Prize distribution math verified: 50% + 30% + 20% = 100%. No overflow risk identified (u64 with reasonable bounds). One Low finding: tournament.maxParticipants is not enforced at the join instruction level — the check exists in off-chain code only.

staking

Basic lockup and reward mechanics are present. Reward calculation uses a fixed APR constant in code rather than a governance-adjustable parameter. Any APR adjustment requires a program upgrade. Low priority but worth noting for long-term governance.

06 ARCHITECTURE REVIEW

6.1 Service Architecture

The monorepo structure with Turborepo is well-organised. Service boundaries are logical. The choice of Fastify over Express is good — schema validation at the framework level catches a lot of errors that would otherwise leak to the database layer.

One structural concern: the NATS event bus is used for inter-service communication, but event publish failures are universally swallowed (try-catch with no retry). If NATS is unavailable, services complete their primary operation and silently drop the event. For the training trigger specifically — if the battle-completed event is dropped, no post-battle training job is queued and the agent never learns from that battle. This is a silent degradation that will not show up in error logs.

Recommendation: implement at minimum a dead-letter pattern — failed publishes should be written to a database table and retried. This does not require changing the happy path.

6.2 Database Schema

The Prisma schema is coherent. Foreign keys are correctly defined. Most critical queries have appropriate indexes. Three index gaps identified:

1. Agent(userId, isRetired) — missing compound index. The "list my agents" query does a full table scan on large deployments.
2. AIModel(agentId, isActive) — missing compound. Model lookup during inference hits this path on every combat tick.
3. AgentMemory(agentId, type) — missing compound. Memory retrieval by type does a filtered scan.

For current user counts these are not causing visible problems. At 10,000+ agents they will.

6.3 Shared Types Inconsistency

The shared-types package defines ClanType as 'CYBER' | 'BIO' | 'ARCANE' | 'MECH' | 'SHADOW'. The Prisma schema (database truth) defines ClanType as 'ZEROG' | 'BASE' | 'SOLANA'. These are completely different enumerations. Any TypeScript code that imports from shared-types and uses ClanType will be operating with wrong values relative to the database. This is a latent bug that will produce confusing runtime errors when real users hit the affected code paths.

Recommendation: delete the ClanType definition from shared-types. Import it from the generated Prisma client instead. Single source of truth, no duplication.

6.4 WebSocket Implementation

H-02 WebSocket battle broadcast not implemented

Impact: The battle-service registers a WebSocket handler for real-time combat streaming. The handler is registered but the broadcast logic (the part that actually sends per-tick battle state updates to connected clients) is not implemented — the function body contains a TODO comment. The gateway correctly proxies WebSocket connections. The handshake succeeds. But no data is ever pushed to the client during a battle.

Fix: Implement the broadcast loop that pushes per-tick state to connected clients. This is the difference between real-time and polling-based combat.

6.5 Inference Service

The 0G Compute integration is correctly implemented. The OpenAI-compatible client is properly configured with the 0G Compute Router base URL. tool_choice: "required" enforcement is in place and working — we verified that malformed responses are rejected and the fallback fires. The 5-second timeout is correctly applied.

One observation: the inference cache (Redis, 1 s TTL) deduplicates calls with identical battle state. The 73% hit rate is reasonable but suggests significant redundant inference calls are still happening. This is probably intentional (avoid stale decisions) but worth monitoring as inference costs scale.

6.6 Memory Service

The four-tier memory architecture (Redis working memory !’ PostgreSQL/Qdrant episodic !’ Qdrant semantic !’ 0G Storage procedural) is well-designed and the implementation largely matches the design. Two gaps:

- The memory compact function runs synchronously on the battle cleanup path. For agents approaching 500 records, this adds 2–7 seconds to post-battle processing. Should be queued as a background job.
- Qdrant collection is initialised with COSINE similarity, which is correct for BGE-M3 embeddings. Verified — no issue with the choice.

07 FINDINGS SUMMARY

CRITICAL — Must fix before wager-enabled launch

C-01 POST /battles/:id/end is unauthenticated
Impact: Any caller can end any battle and declare any winner, triggering real $ARENA escrow settlement.
Fix: Add X-Service-Key authentication. Validate winnerId against battle record.
Effort: 1–2 hours

C-02 inft-service calls non-existent ABI method (agentIdToTokenId)
Impact: All INFT operations fail silently. inftTokenId is never populated. INFT feature is entirely non-functional.
Fix: Correct the method name to the existing reverse mapping, or add the missing mapping function to the contract.
Effort: 2–4 hours

HIGH — Fix before public launch

H-01 Matchmaking silent failure at 500+ concurrent agents
Impact: Queue joins fail without returning a 503. Add explicit capacity checks and meaningful errors.

H-02 WebSocket battle broadcast not implemented
Impact: Per-tick real-time streaming does not work. Frontend must fall back to polling.

H-03 INTERNAL_SERVICE_SECRET not required at startup
Impact: If env var is unset, all internal service routes are accessible without authentication.

H-04 INFT clone() does not verify token ownership
Impact: Authorized executors can clone any INFT token, not just tokens they own.

H-05 sealedKey stored as unconstrained string on-chain
Impact: No enforcement that sealedKey contains only a hash. Potential for sensitive data stored permanently on a public chain.

H-06 Escrow cancel() has no time-lock
Impact: Either party can cancel an in-progress wager battle before results are submitted, griefing the other participant.

MEDIUM — Fix within 30 days of launch

- M-01: NATS event publish failures are silently swallowed with no retry
- M-02: Missing compound indexes on Agent, AIModel, AgentMemory tables
- M-03: ClanType enum mismatch between shared-types and Prisma schema
- M-04: Access token not invalidated on logout (15-minute exposure window)
- M-05: Memory compact runs synchronously on battle cleanup path
- M-06: agentId parameter not validated as UUID before database query
- M-07: updateMemoryRoot() and updateModelRoot() emit no events
- M-08: AgentRegistry.sol has no deregistration path
- M-09: Tournament maxParticipants not enforced in Anchor program
- M-10: Matchmaking waitTimeMs reflects elapsed time, not estimated remaining (misleading to clients)
- M-11: training-service job queue has no dead-letter handling for failed jobs

LOW — Schedule for next development cycle

18 low-severity findings covering: missing request logging on internal routes, CORS R2 bucket URL not covering 0G Storage CDN, staking APR hardcoded in program rather than governance-adjustable, several TODO comments left in production-facing code paths, minor input sanitisation gaps on non-security-sensitive fields, inconsistent HTTP status codes (some 200 responses where 201 is correct), and contract address duplication between documentation and .env.example.

INFORMATIONAL

9 informational observations covering: architectural patterns that are correct but worth documenting formally, dependency versions that are current now but will need monitoring, Render cold-start behaviour and its impact

on p99 latency, and recommendations for load testing tooling if the team wants to establish a performance regression baseline.

08 RECOMMENDATIONS

Priority 1 — Before wager battles go live (block on these)

1. Fix C-01 and C-02. These are not complex fixes — they are each a few lines of code. C-01 is the most urgent: an unauthenticated battle-end endpoint on a platform handling real token escrow settlements is an unacceptable risk. Fix it this week.
2. Fix H-03. Add a startup assertion that INTERNAL_SERVICE_SECRET is set and is a non-empty string of sufficient length. The service should refuse to start without it. This is a one-liner.
3. Fix H-06. Add a minimum 10-minute lock period to the escrow cancel() instruction. Require admin signature to cancel within the lock window.

Priority 2 — 30-day post-launch window

1. Implement a dead-letter table for NATS publish failures. This is the most impactful architectural improvement available. Silent event loss will create subtle, hard-to-diagnose data consistency bugs over time.
2. Fix the ClanType enum (M-03). This will cause production bugs when real users hit code paths that use shared-types ClanType. The fix is a one-time deletion of the wrong definition.
3. Add the three missing database indexes (M-02). They do not hurt current performance but will cause visible degradation at moderate scale.
4. Implement WebSocket battle broadcasting (H-02). This is a larger effort but it is the difference between real-time and polling-based combat. Worth prioritising for product quality.

Priority 3 — Architecture improvements (60–90 day horizon)

1. Make 0G Storage uploads asynchronous in the battle-end pipeline. This single change cuts the user-facing battle-end latency from 3.4 s average to under 1 s.
2. Implement token blocklisting for logout (M-04). Requires a Redis SET of invalidated JTIs with TTL matching access token lifetime. Standard pattern, well-documented.
3. Move memory compact to a background job queue (M-05). Use the existing NATS/BullMQ infrastructure rather than running it inline.
4. Explore a fallback inference provider for the 3.2% timeout case. The current fallback (hardcoded defensive action) is a safe degradation but a secondary provider call before throwing the fallback would maintain agent autonomy through 0G network hiccups.
5. Consider a WebSocket heartbeat and reconnect strategy on the client side, independent of the broadcast implementation. Users on mobile or flaky connections will disconnect mid-battle.

09 CONCLUSION

The AI Arena platform is more complete than most systems we audit at this stage. The 0G ecosystem integration is genuinely well-done — the pattern of content-addressing every training artifact and anchoring hashes on-chain is not something we see implemented correctly very often. The Solana escrow state machine is clean. The Fastify microservice architecture is sensible and the Prisma schema is coherent.

The two critical findings are serious but they are not fundamental design problems. POST /battles/:id/end being unauthenticated is a deployment oversight, not an architectural flaw — the comment in the code even acknowledges it needs to be fixed. The INFT ABI mismatch is a one-line bug. Neither requires rethinking anything.

What the platform needs before a high-stakes public launch is a focused two-week hardening sprint targeting the critical and high findings, followed by the medium-priority fixes rolling out over the following month. That is a realistic timeline.

The foundation is solid. Finish the build.

Meridian Protocol Labs

J. Hartmann — Senior Auditor
S. Okafor — Contract Auditor
R. Petrov — Security Analyst

Report Reference: MPL-2026-0312 · Issued: May 27, 2026